Cụm 8 - Case Studies

Mục lục

8.1 Tổng quan Cụm 8: Case Studies

> **Tóm tắt một dòng**: 7 cụm trước cung cấp building block. Cụm này áp tất cả vào 2 case thực để bạn thấy cách lập luận “given context X, why this architecture not that” - skill thực sự của architect.

Vì sao case study?

Lý thuyết kiến trúc dễ học, áp dụng khó. Cụm 1-7 cho bạn vocabulary và principles. Nhưng câu hỏi thực sự khi làm architect là:

- “Hệ này nên monolith hay microservices?”
- “Database nào? SQL hay NoSQL?”
- “Communication nào? Sync HTTP hay async event?”
- “Architecture style nào best fit?”

Câu trả lời tuỳ context. Case study đi qua **lập luận end-to-end** cho hai context khác nhau, show cách architect thinks.

2 Case Studies

Case 1: UAMS, University Academic Management System

Context:

- Trường đại học multi-program (undergraduate, postgraduate).
- Centralized system manage student, course, grade.
- Regulatory compliance (data privacy, audit trail).
- Scale: 50k student peak (registration week).

Focus: structured business processes, ACID requirements, internal users.

Case 2: Smart City Traffic Incident Detection

Context:

- City government deploy system phát hiện sự cố giao thông real-time.
- Data sources: camera, sensor, GPS, citizen mobile reports.
- Notify operator, emergency services, navigation systems.
- Scale: thousands sensors, millions data points/hour, sub-second response.

Focus: high-throughput streaming, real-time, heterogeneous data.

Bài tập

Cuối cụm có bộ exercises để bạn tự apply.

Cách áp dụng framework cho mỗi case

Mỗi case sẽ đi qua 6 bước (synthesizing các cụm trước):

Bước 1: Hiểu domain + functional requirements (Cụm 1)

Đào ra hệ làm gì, ai dùng, business context.

Bước 2: Identify Quality Attributes (Cụm 4)

Top 5-7 QA priority. Operationalize.

Bước 3: Cân nhắc Style, Monolithic vs Distributed (Cụm 5.2)

Default monolithic; chỉ distributed nếu justify.

Bước 4: Choose specific Architecture Style (Cụm 5-6)

Match QA priorities với style trade-offs.

Bước 5: Component decomposition (Cụm 2.3 SRP + Cụm 3.4 + Cụm 4.4)

Bounded context, module boundary.

Bước 6: Document (Cụm 7)

C4 Context + Container, key sequence, ADR for important decisions.

Sau mỗi case sẽ có "What we'd do differently", phần phản biện.

Sản phẩm cuối case

Mỗi case xuất ra:

1. **System Context diagram** (C4 Level 1).
2. **Container diagram** (C4 Level 2).
3. **2-3 sequence diagram** cho critical flows.
4. **3-5 ADR** cho key decisions.
5. **QA scorecard**: bảng QA vs measurement.
6. **Trade-off reflection**: "đã hy sinh gì để đạt gì".

Khi bạn làm xong, output này là *template* cho architecture doc của project bạn.

Tone của cases

Cases viết theo phong cách "consulting walk-through", như consultant trình bày cho client. Bạn đọc như đang sit-in một consulting engagement.

Mỗi case dài ~3500-5000 chữ, đọc 1.5-2h. Đáng dành thời gian.

Bài trong cụm

- **8.2 UAMS**: Academic Management System.
- **8.3 Smart City Traffic**: Real-time incident detection.
- **8.4 Exercise Set**: 5 bài tập tự practice với hint.

Vào Bài 8.2: UAMS để bắt đầu.

8.2 UAMS, University Academic Management System

> **Tóm tắt một dòng**: Case study lập luận end-to-end cho hệ academic management. Conclusion - Service-based architecture với 6 services, shared PostgreSQL với schema-per-service, sync HTTP cho internal call, sync API + sync admin UI. Tổng cost dự kiến $1500/tháng cho 50k students.

1. Domain và Functional Requirements

Tổ chức bối cảnh

Một trường đại học chạy đa chương trình (undergraduate, postgraduate). Quyết định xây dựng centralized Academic Management System để chuẩn hoá operations và đảm bảo compliance regulatory.

Core features

- **Student enrollment** + profile management.
- **Course + curriculum management**.
- **Semester registration** (sinh viên đăng ký môn).
- **Grade submission** + GPA calculation.
- **Role-based access control** (student, instructor, registrar, admin).

Users

- **Students** (peak 50k, average 30k active).
- **Instructors** (~2000).
- **Registrars** (~50 staff).
- **Admins** (~10).

Usage patterns

- **Regular**: spread throughout semester, modest load (100-500 req/min).
- **Registration week**: 10x normal. 50k student đăng ký môn trong 2-3 ngày.
- **Grade submission week**: bursty từ instructor side.

2. Identify Quality Attributes

Workshop với stakeholder (Registrar Director, IT Director, Dean of Studies):

Top 7 QA

Priority	QA	Target
Must	Data integrity	100% (registration không có double-booking, grade chính xác)
Must	Security	Compliance regulatory (FERPA equivalent). Audit log mọi grade change

Priority	QA	Target
Must	Availability	99.5% (4 hours downtime/month acceptable, except registration week)
Must	Auditability	Full trace của ai sửa grade/enrollment khi nào
Should	Scalability	Handle 10x load in registration week
Should	Maintainability	New feature ship trong 1-2 sprint
Should	Cost	< $3000/month infrastructure cho 50k students

Implicit characteristics

- Privacy (data sinh viên).
- Performance acceptable (page load < 2s).
- Onboard new staff < 1 week (admin UI usable).

Operationalize

QA	Metric	Tool
Data integrity	0 inconsistency in audit (quarterly)	DB constraint, automated reconciliation
Security	OWASP Top 10 quarterly pen-test	External pen-test
Availability	Uptime % from monitoring	Datadog/UptimeRobot
Auditability	100% grade changes have audit log	DB trigger + audit table
Scalability	Handle 50k concurrent registration	Load test pre-registration week
Maintainability	Mean PR-to-prod time < 1 week	Git analytics
Cost	Monthly cloud bill	Cloud billing dashboard

3. Monolithic vs Distributed

Considerations

- **Team size**: ~15 engineers (internal IT). Not huge.
- **Domain**: Medium complexity (~6 bounded contexts).
- **Scale**: 50k users peak. Significant but not enormous.
- **Regulatory**: separate audit log easier in microservices, but monolithic with proper modules also OK.

Decision

Service-based (4-12 coarse services, shared DB per domain). Reasons:

- Team ≈ 15 □ not enough for full microservices (need 50+).
- 6 bounded contexts □ service-based natural fit.
- 10x scaling spike □ service-based scales OK (better than monolith, less complex than microservices).
- Operational team không có K8s expertise □ microservices nặng.

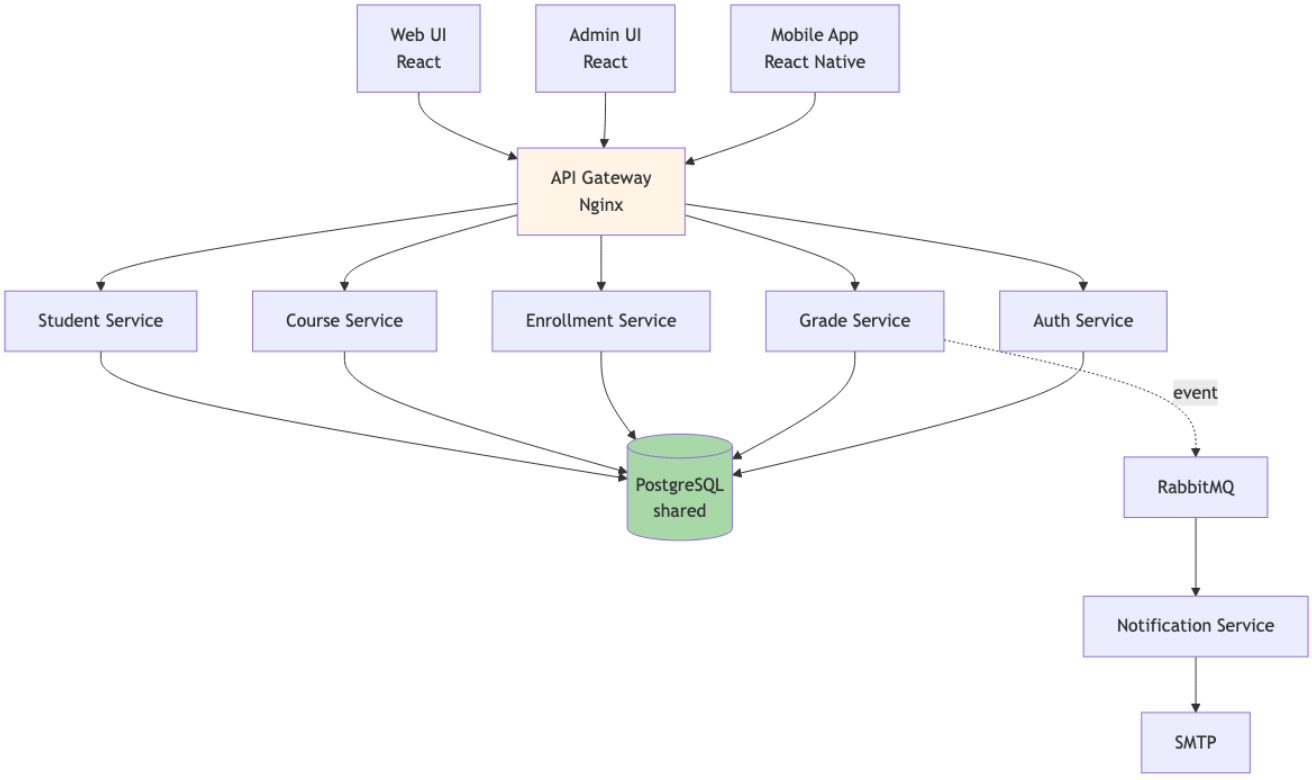
Justify cost-benefit:

- Service-based ops cost ~$1500/month vs monolith $500/month vs microservices $5000/month.
- Microservices không justify ROI cho team 15 + scale này.

4. Architecture Style Selection

Style choice: Service-based + selective async

- **Sync HTTP (REST)** for internal service calls.
- **Async event** (RabbitMQ) for cross-context notifications (vd: grade submitted □ notify student email).
- **Shared PostgreSQL** with schema-per-service.



Hình 1: Sơ đồ 43

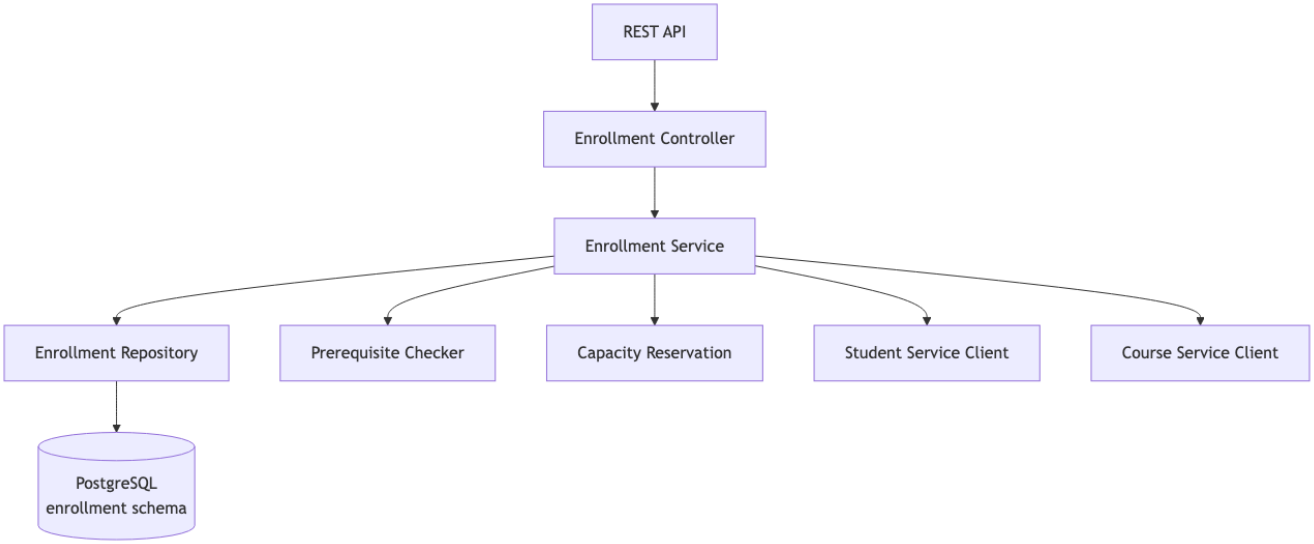
Bounded contexts □ services

Service	Bounded Context	Owns Entities
Student Service	Student domain	Student profile, enrollment history
Course Service	Catalog	Course, Curriculum, Section
Enrollment Service	Registration	Registration, Schedule
Grade Service	Assessment	Grade, GPA calculation
Auth Service	Identity	User, Role, Permission
Notification Service	Communication	Email/SMS queue

6 services, total 15-engineer team (2-3 engineers per service).

5. Component Decomposition

Inside Enrollment Service (example)



Hình 2: Sơ đồ 44

Components inside service follow layered (Cụm 5.3).

6. Documenting Key Decisions (ADRs)

ADR-001: Use Service-based over Microservices

```
## Context
Team 15 engineers, 6 bounded contexts, 50k user peak, ops team không có K8s.

## Decision
Service-based với 6 coarse services + shared PostgreSQL.

## Alternatives Considered
- Monolith: pain với deploy bottleneck dự kiến trong 2 năm.
- Microservices: cost ops 3-5x, team không ready.
```

```
## Consequences
- Lợi: team velocity, deploy independent, manageable ops.
- Hại: chưa có full team independence như microservices.
```

ADR-002: Shared PostgreSQL với i schema-per-service

```
## Decision
Single PostgreSQL instance, schema-per-service (student_svc, course_svc, ...).

## Rationale
- Cost: 1 DB instance vs 6 instances.
- Cross-service query: support khi cần (vd: report tổng hợp).
- Compliance audit: single DB easier để audit.
- Trade-off: phải discipline schema migration coordination.
```

ADR-003: Sync HTTP for inter-service

```
## Decision
Sync HTTP (REST) for service-to-service calls, async RabbitMQ for notifications.

## Rationale
- Sync: simple, debug dễ, fit cho most user-facing flows.
- Async: notification không cần real-time, decoupling email service.

## Consequences
- Cascade failure risk if service down: mitigate by circuit breaker (Resilience4j).
- Latency: 3-5 hops max for any user request, sub-1s achievable.
```

ADR-004: Audit log via DB trigger

```
## Decision
PostgreSQL trigger on grade/enrollment tables → write to audit_log table.

## Rationale
- Compliance: full trace required.
- DB-level capture: cannot bypass via app bug.
- Trade-off: DB trigger complexity, performance overhead ~5%.

## Alternatives Considered
- Application-level logging: rejected (can be bypassed).
- CDC (Change Data Capture): rejected (extra infrastructure complexity).
```

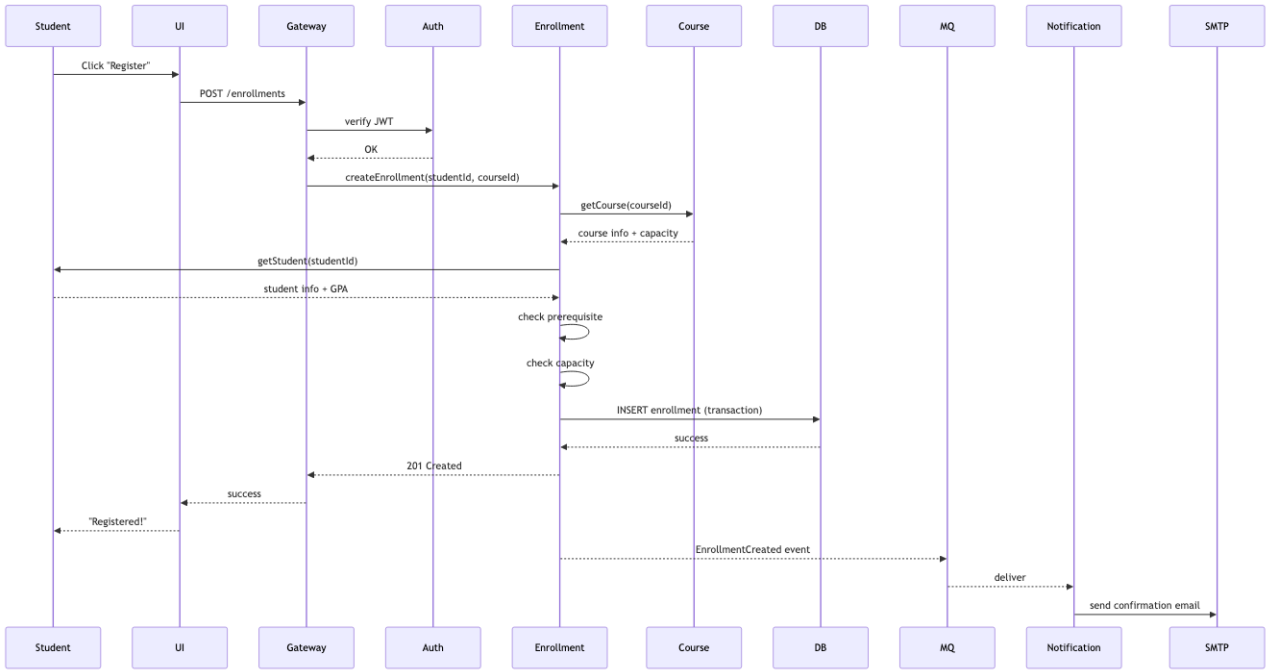
ADR-005: Auto-scale Enrollment Service for registration week

```
## Decision
Auto-scale Enrollment Service (3 → 30 instances) during registration week.
Other services stay 2-3 instances.

## Rationale
- 10x load only on Enrollment during registration.
- Cost optimization: scale only what's needed.
```

7. Critical Flows

Flow 1: Student registers for course



Hình 3: Sơ đồ 45

Flow 2: Instructor submits grade

(similar pattern, brief)

8. QA Scorecard

QA	Target	How achieved
Data integrity	100%	DB constraints, transaction, automated reconciliation job
Security	OWASP compliant	Auth service, RBAC, prepared statements (SQL injection), HTTPS, secret in Vault
Availability	99.5%	Multi-AZ, RDS HA, service replicas, circuit breakers
Auditability	100% grade changes	DB trigger + audit_log table

QA	Target	How achieved
Scalability	Handle 50k peak	Auto-scale Enrollment, read replicas during peak
Maintainability	PR-to-prod < 1 week	CI/CD per service, contract tests
Cost	< $3000/month	$1500 baseline + $1000 during peak weeks = $1500-2500/month average

All targets met.

9. Trade-off Reflection

Hy sinh:

- **Full microservices independence**: 6 services share DB, không thể database-per-service migrate hoàn toàn isolated.
- **Polyglot tech stack**: all services Java/Spring (team uniform). Không thể chọn Python cho ML service nếu cần sau này (would require refactor).
- **Sub-100ms latency**: sync calls accumulate. p99 200-500ms typical, acceptable cho academic use.

Đạt:

- **Pragmatic ops**: 1 ops engineer can handle 6 services + 1 DB.
- **Team velocity**: each team owns 1-2 services, deploy independently.
- **Compliance**: audit + RBAC built into architecture.
- **Cost-effective**: $1500-2500/month for 50k students.

10. What we'd do differently

Honest reflection:

1. **Add event sourcing for Grade**: Audit log good but reconstructing historical grade state hard. Event sourcing for grade entity would simplify.
2. **API versioning từ ngày 1**: Currently API not versioned. When mobile app released, breaking changes painful.
3. **Async cho slow operations**: Bulk grade upload by instructor □ sync HTTP timeout. Should be async with progress tracking from start.
4. **Multi-region for DR**: Single region acceptable for academic, but disaster recovery plan thiếu. Should have replica in second region.

Tóm tắt

UAMS case demonstrates:

- **Service-based** chosen over monolith (team velocity) vs microservices (ops cost).
- **Shared DB schema-per-service** balance isolation + cost.
- **Sync HTTP** for user flows, **async event** for notifications.
- **Audit via DB trigger** for compliance.
- **Auto-scale** specific service for predictable load spike.

Pragmatic, not perfect. Apt for context.

Bài tiếp: Smart City Traffic Detection, context completely different.

8.3 Smart City Traffic Incident Detection

> **Tóm tắt một dòng**: Case study lập luận cho hệ phát hiện sự cố giao thông real-time. Conclusion - Event-Driven Architecture với Kafka pipeline, microservices style cho detection logic, time-series DB cho historical analytics. Khác hoàn toàn UAMS vì context khác.

1. Domain và Functional Requirements

Background

City government deploy Smart Traffic Incident Detection System để cải thiện an toàn đường + giảm congestion.

Data sources (heterogeneous)

- **Roadside cameras** (~5000 cameras, 1 frame/giây mỗi camera).
- **Traffic sensors** (~10,000 sensors đo speed + volume, update mỗi 5 giây).
- **GPS data** từ public buses + taxis (~3000 vehicles, ping mỗi 10 giây).
- **Citizen reports** (mobile app, sporadic, ~100 reports/giờ).

Functional requirements

- Ingest data heterogeneous, high-volume.
- Clean + normalize.
- Detect incidents real-time: accidents, stalled vehicles, congestion (rule-based + AI/ML).
- Generate alerts với severity levels.
- Store data cho historical analysis (~7 years).
- Notify traffic operators, emergency services, navigation systems.

Scale calculation

- Cameras: 5000 × 1 frame/s = 5000 events/s.
- Sensors: 10000 × 1 update / 5s = 2000 events/s.
- GPS: 3000 × 1 ping / 10s = 300 events/s.
- Citizen: ~0.03 events/s.

Total: **~7300 events/s** sustained, peak 15-20k events/s.

Per day: ~700 million events. Per year: ~250 billion. Storage massive.

2. Identify Quality Attributes

Top 7 QA

Priority	QA	Target
Must	Scalability	Handle 20k events/s sustained, scale to 50k events/s
Must	Low latency (detection)	Incident detected within 30s of event
Must	Low latency (alert)	Alert delivered within 10s of detection

Priority	QA	Target
Must	Availability	99.9% (detection must not miss critical incidents)
Must	Extensibility	Add new detection algorithm without redeploy core
Should	Reliability	No data loss in pipeline
Should	Cost	< $20k/month for current scale

Operationalize

QA	Metric	Tool
Scalability	Events/s throughput, lag	Kafka metrics, Datadog
Detection latency	Time from event to detection	Distributed tracing
Alert latency	Time from detection to notification	Logs
Availability	Uptime %	Multi-region health check
Extensibility	Time to deploy new algorithm	Git analytics
Reliability	Message loss rate	Kafka offset monitoring
Cost	Monthly cloud bill	Cloud billing

3. Monolithic vs Distributed

Considerations

- **Scale**: 7-20k events/s, monolithic single instance không feasible.
- **Heterogeneous sources**: different ingestion logic, parallel.
- **Real-time detection**: cần stream processing.
- **Extensibility**: add detection algo dễ plug-in style.

Decision

Distributed, Event-Driven Architecture (Cụm 6.4) overlay với **Microservices** (Cụm 6.3) cho detection services.

Justify:

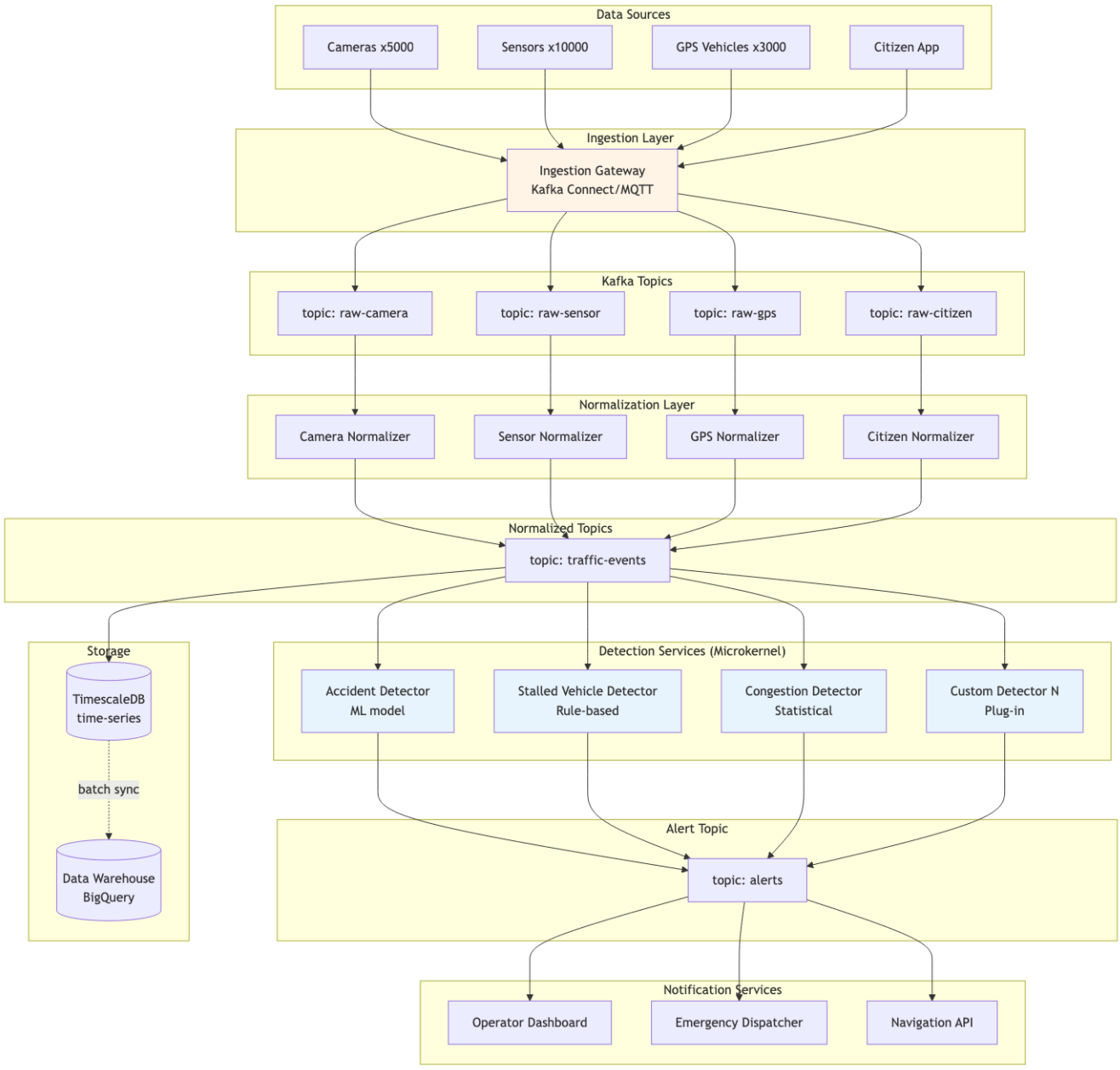
- Scale + throughput requirements distributed required.
- Async detection chain event-driven natural.
- Extensibility for new algorithm microservices + Microkernel pattern.

4. Architecture Style

Style mix

- **Event-Driven (Cụm 6.4)**: backbone với Kafka.
- **Microservices (Cụm 6.3)**: fine-grained detection services.
- **Microkernel (Cụm 5.5)**: detection services là plug-ins to a “detection core”, easy add new algorithm.
- **Pipeline (Cụm 5.4)**: ingest normalize detect là pipeline.

Multiple styles compose. Architect chọn fit-for-purpose per layer.



Hình 4: Sơ đồ 46

5. Component Decomposition

Ingestion services (1 per source type)

- Camera Ingester: MQTT broker □ Kafka.
- Sensor Ingester: HTTP POST endpoint □ Kafka.
- GPS Ingester: TCP socket (NMEA protocol) □ Kafka.
- Citizen Ingester: HTTPS REST API □ Kafka.

Normalizer services

Mỗi normalizer:

- Consume raw Kafka topic.
- Validate, clean, normalize to common schema.
- Publish to `traffic-events` topic.

Common schema:

```
{
  "event_id": "uuid",
  "timestamp": "ISO 8601",
  "source_type": "camera|sensor|gps|citizen",
  "source_id": "string",
  "location": {"lat": ..., "lon": ..., "road_id": "..."},
  "data": {...source-specific...},
  "metadata": {...}
}
```

Detection services (microkernel plug-ins)

Mỗi detector:

- Subscribes `traffic-events` topic (filter by location/source).
- Apply detection logic (rule, statistical, ML).
- Publish `IncidentDetected` event to `alerts` topic.

New detector = new microservice subscribing topic. Core không thay đổi.

Storage

- **TimescaleDB**: hot data, last 30 days, real-time query.
- **BigQuery**: cold data, 7 years, analytics.

Pipeline batch sync TimescaleDB □ BigQuery daily.

6. Key ADRs

ADR-001: Kafka as event backbone

```
## Decision
Apache Kafka as event broker for entire pipeline.
```

```
## Rationale
- Throughput: handles 20k+ events/s easily.
- Replay: critical for ML model retraining and debugging.
- Schema registry support.
- Industry standard for streaming.

## Alternatives
- RabbitMQ: not designed for stream replay, lower throughput.
- AWS Kinesis: vendor lock, similar capability.
- Pulsar: newer, less ecosystem.

## Consequences
- Ops complexity (Kafka cluster management), invest in tooling.
- Need schema registry (Confluent or Apicurio).
```

ADR-002: Microkernel for detection services

```
## Decision
Detection logic implemented as plug-in microservices that subscribe to traffic-events topic.

## Rationale
- New detection algorithms common (city evolves rules).
- Each algorithm runs independently, fault isolation.
- Can be deployed by different teams.

## Consequences
- Plug-in API (Kafka schema) must be stable.
- Discovery: each algorithm registers its capabilities.
```

ADR-003: TimescaleDB + BigQuery split

```
## Decision
TimescaleDB (PostgreSQL extension) for hot 30 days. BigQuery for 7-year history.

## Rationale
- Real-time dashboards query last 30 days → need low-latency time-series DB.
- 7-year analytics → batch jobs OK → BigQuery cost-effective for cold data.
- Cost: TimescaleDB ~$1500/month for 30 days. BigQuery storage cheap, query pay-per-use.

## Consequences
- Two systems to manage.
- Sync job complexity.
- Cross-DB query not possible (acceptable).
```

ADR-004: At-least-once delivery with idempotent consumers

```
## Decision
Kafka at-least-once delivery. Detectors implement idempotency by event_id.

## Rationale
- Exactly-once expensive and complex.
- Most detection logic naturally idempotent (same event, same detection).
- Alert deduplication at notification layer.

## Consequences
- All detectors must implement idempotency. Code review enforce.
```

ADR-005: Geo-sharded Kafka topics

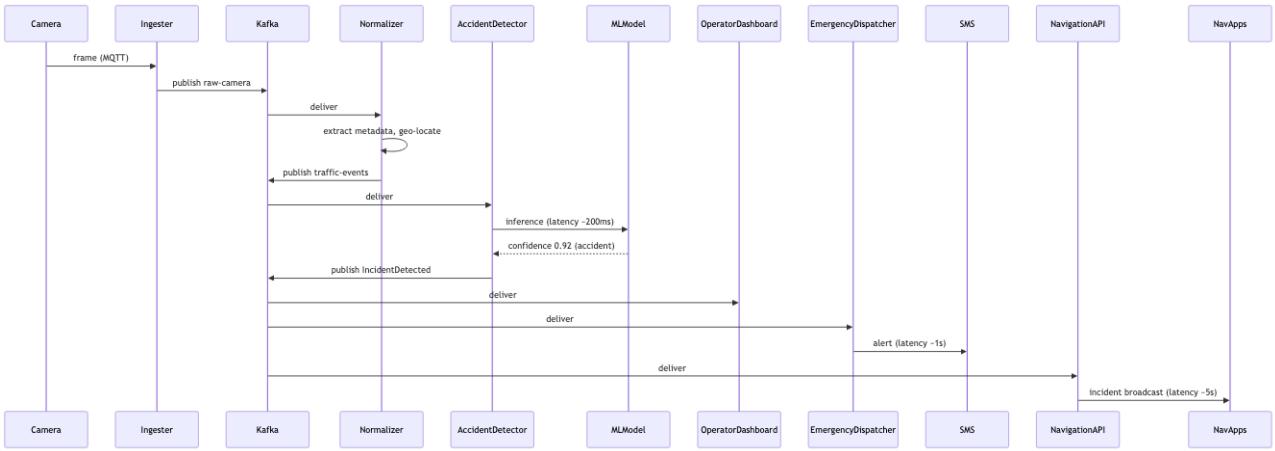
```
## Decision
Kafka topics partitioned by geographic region (vd: district 1-12).

## Rationale
- Locality: detection algorithms can subscribe to specific districts.
- Scale: parallelize processing by partition.
- Future: regional clusters if needed.

## Consequences
- Routing logic in ingester (which partition).
- Re-shard rare but complex.
```

7. Critical Flow

Detection of accident from camera frame



Hình 5: Sơ đồ 47

End-to-end: camera frame to alert ~15-20s. Within target.

8. QA Scorecard

QA	Target	How achieved
Scalability	20k events/s	Kafka partition, horizontal scale detector
Detection latency	30s	Streaming pipeline (vs batch)
Alert latency	10s	Kafka consumer group with low-latency config
Availability	99.9%	Multi-AZ Kafka, replica consumers, multi-region failover
Extensibility	New algorithm in days	Microkernel pattern, well-defined event schema
Reliability	No data loss	At-least-once + idempotent consumers
Cost	< $20k/month	Auto-scale, BigQuery for cold data

9. Trade-off Reflection

Hy sinh:

- **Strong consistency**: events arrive out of order across partition (per-partition ordered). Detection algorithms handle.
- **Operational complexity**: Kafka cluster + many microservices + 2 storage systems = need experienced SRE team.
- **Cost**: $15-20k/month higher than UAMS but justify by scale.

Đạt:

- **Scalable to 50k+ events/s** with horizontal scaling.
- **Sub-30s detection** real-time.
- **Plug-in extensibility** for new algorithms.
- **7-year audit** via BigQuery.

10. So sánh UAMS vs Smart City

Aspect	UAMS	Smart City
Style	Service-based	Event-Driven + Microservices + Microkernel
DB	Shared PostgreSQL	TimescaleDB + BigQuery
Communication	Sync HTTP + async event	Async event chỉ
Scale	50k peak users	20k events/s sustained
Latency target	< 2s page load	< 30s detection, < 10s alert
Cost	$1500-2500/month	$15-20k/month
Team needed	15 engineers	25-30 engineers + 5 ML + 3 SRE

Architecture khác hoàn toàn vì *context khác*. Same lecturer's principles, different applications.

Tóm tắt

Smart City case demonstrates:

- **EDA + Microservices + Microkernel** compose cho high-scale real-time.

- **Kafka** as backbone for streaming.
- **Pipeline pattern** in ingest □ normalize □ detect.
- **Microkernel** for extensibility (add algorithms).
- **TimescaleDB + BigQuery** for hot/cold data split.

Both UAMS và Smart City đều “correct”, vì serve different contexts. Skill: chọn đúng combination.

Bài tiếp: Exercise Set, bạn tự practice.

8.4 Bài tập tổng hợp

> **Tóm tắt một dòng**: 5 bài tập tự practice. Mỗi bài có context, requirements, và hints để bạn lập luận theo framework từ Cụm 1-7. Không có "đáp án đúng" - chỉ có "đáp án hợp lý cho context".

Cách làm

Cho mỗi bài tập:

1. **Đọc context** + functional requirements.
2. **Identify Top 5-7 Quality Attributes** (Cụm 4).
3. **Quyết định monolithic vs distributed** (Cụm 5.2).
4. **Choose architecture style** (Cụm 5-6).
5. **Decompose components** (Cụm 4.4, 3.4).
6. **Draft 3-5 ADRs** cho key decisions (Cụm 7.2).
7. **Vẽ C4 Context + Container diagrams** (Cụm 7.2).
8. **Identify trade-offs hy sinh** (Cụm 3.3).

Time budget: 2-3 giờ mỗi bài.

Tốt nhất là làm với 1-2 đồng nghiệp + present cho nhau. SA là kỹ năng *socialized*, debate giúp identify weak reasoning.

Bài 1: Online Food Delivery Platform

Context

Startup Việt Nam build platform giao đồ ăn (như GrabFood, ShopeeFood).

- Phase 1 (6 tháng): launch tại HCM, ~1000 restaurants, ~50k customer, ~500 driver.
- Phase 2 (12 tháng): expand 5 thành phố lớn, 10x scale.
- Phase 3 (24 tháng): national, 100x scale phase 1.

Functional requirements

- Customer browse restaurant + menu, place order.
- Restaurant receive order, confirm/reject.
- Driver được assigned, pickup, deliver.
- Real-time tracking driver location for customer.
- Payment (digital wallet, COD).
- Rating + review.

Constraints

- Initial team: 8 engineers.
- Budget: < $5k/month infrastructure phase 1.

Hints

- **Scale trajectory**: design cho phase 1, planned migration to phase 2.
- **Real-time tracking**: WebSocket + location streaming.

- **Order workflow**: state machine (placed □ confirmed □ preparing □ ready □ picked □ delivered).
- **Multi-sided platform**: 3 user types (customer, restaurant, driver), consider BFF pattern.

Things to consider

- Monolithic phase 1, migrate phase 2? Hay service-based từ đầu?
- DB: PostgreSQL? Add Redis? Add Kafka?
- Payment: build hay integrate (Stripe/local gateway)?
- Notification: push notification setup?

Output expected

- C4 Context + Container.
- Top 5 QA + targets.
- 4-5 ADRs.
- Trade-off reflection.

Bài 2: B2B SaaS for Property Management

Context

SaaS giúp building owner quản lý apartment complex (như Yardi, RealPage but VN-focused).

- Khách hàng: building owner / management company (100-2000 apartments per customer).
- Target: 50 customers in year 1, 500 in year 3.

Functional requirements

- Tenant management (lease, contract, profile).
- Rent collection (recurring billing, payment tracking).
- Maintenance ticket (tenant submit □ assigned to staff □ resolve).
- Financial reporting (collected, outstanding, occupancy rate).
- Customer admin portal + Tenant mobile app.

Constraints

- Multi-tenancy (khách hàng A không thấy data khách hàng B).
- Customization: each building can have custom rules (vd: late fee policy).
- Compliance: GDPR-like privacy regulation.

Hints

- **Multi-tenancy**: shared DB + tenant_id column? Database-per-tenant? Schema-per-tenant?
- **Customization**: rule engine? Plug-in?
- **Reporting**: separate read replica? CQRS?
- **Mobile app**: BFF for tenant?

Things to consider

- Scale 50-500 customers, service-based likely sufficient.
- Multi-tenancy strategy có ảnh hưởng lớn đến architecture.
- Compliance audit log.

Output expected

- Same as Bài 1.
- Bonus: design multi-tenancy schema in detail.

Bài 3: Real-time Stock Trading Platform

Context

Startup fintech build platform giao dịch chứng khoán (Robinhood-style).

- Target users: 100k retail trader Vietnam.
- Peak hours: 9:00-15:00 weekdays.
- Connect to local exchange (HOSE, HNX) via FIX protocol.

Functional requirements

- View real-time price ticker.
- Place buy/sell order.
- Portfolio tracking.
- Historical price chart (1-min, 5-min, daily candles).
- News feed.
- Account & money transfer.

Constraints

- Compliance: SBV regulation, audit log mọi transaction.
- Latency critical: order execution < 100ms.
- Data integrity: 100% accurate (financial).
- Uptime peak hours: 99.99%.

Hints

- **Price ticker**: WebSocket streaming. Pub-sub pattern. Redis pub/sub or Kafka?
- **Order matching engine**: critical path, likely in-house cho fairness/latency.
- **Historical data**: time-series DB (TimescaleDB, InfluxDB).
- **Compliance**: event sourcing (cho audit + replay)?

Things to consider

- This is “harder” than typical app, financial, real-time, regulated.
- Space-Based architecture có relevant không (for matching engine)?
- Disaster recovery critical.

Output expected

- Same as before + DR/failover plan.
- Critical: explain latency budget breakdown.

Bài 4: IoT Manufacturing Monitoring

Context

Manufacturing company want monitor 50 factories worldwide. Each factory has 200 machines với sensor (temperature, vibration, power, …).

Functional requirements

- Ingest sensor data (10k sensors total, each pings every 5s = 2000 events/s).
- Real-time dashboard cho factory manager.
- Anomaly detection (predictive maintenance).
- Historical analytics (5 year retention).
- Alert system (SMS + email + integration với existing ticketing).

Constraints

- Some factories có poor internet, edge processing cần thiết.
- Latency: anomaly detection within 60s.
- Cost-sensitive (industrial budget, not tech startup).
- Compliance: ISO 27001.

Hints

- **Edge processing**: do detection locally, sync results to central.
- **Cloud + on-prem hybrid**.
- **Time-series storage**: InfluxDB, TimescaleDB.
- **Anomaly detection**: rule + ML.

Things to consider

- Hybrid (edge + cloud) is rare but appropriate here.
- Bandwidth limited □ compress + batch.
- Lambda architecture (batch + stream) possible.

Bài 5: Multi-region E-commerce Platform

Context

Major e-commerce expand từ Vietnam ra Southeast Asia (Thailand, Indonesia, Philippines, Malaysia).

- Current: monolith Vietnam, 10M users.
- Target: multi-region, 50M users across SEA.
- Constraints: comply local data residency law (each country’s data must stay in-country).

Functional requirements

- Catalog (products country-specific + global).
- Cart, checkout, payment (country-specific payment methods).
- Inventory (country-specific warehouses).
- Search (country-specific language).
- Recommendations (cross-region OK).

Constraints

- Latency: page load < 2s in each country.
- Data residency: hard requirement.
- Existing monolith: 5-year-old codebase, can't rewrite.

Hints

- **Migration strategy**: Strangler Fig (Cụm 5.2).
- **Multi-region deployment**: each country has DB; some services shared globally (recommendations).
- **CDN**: critical for asset performance.
- **Data residency**: dictate database location, search index, cache.

Things to consider

- This is the hardest exercise, combine migration + multi-region + compliance.
- Hybrid: legacy monolith + new microservices.
- Plan over 24-36 months.

Reflection questions

Sau khi làm 5 bài:

1. Bài nào dễ nhất? Vì sao?
2. Bài nào khó nhất? Vì sao?
3. Style nào bạn dùng nhiều nhất across exercises? Có pattern không?
4. QA nào hay được prioritize? (security, scalability, …)
5. Quyết định bạn ít confident nhất? Cần học gì thêm?

Câu trả lời tiết lộ strengths + gaps của bạn. Use it to plan further learning.

Discussion với mentor (nếu có)

Show bài tập của bạn cho:

- Senior architect tại công ty.
- Mentor từ tech community.
- Bạn cùng nhóm học.

Hỏi:

- "Đâu là điểm tôi missing?"
- "Bạn would do differently cái nào?"
- "Quyết định nào bạn would push back?"

Feedback từ người có kinh nghiệm thực tế > self-study 10 giờ.

Tóm tắt

- 5 bài tập từ easy (food delivery) đến hard (multi-region migration).
- Apply framework end-to-end.
- Discuss với mentor để identify gap.

Tổng kết Cụm 8 và toàn khoá

Hết Cụm 8. Hết khoá.

Tóm tắt journey:

- **Cụm 1**: framework tư duy.
- **Cụm 2**: foundation principles (SOLID, cohesion-coupling).
- **Cụm 3**: tư duy architect (vs designer).
- **Cụm 4**: cái cần tối ưu (QA).
- **Cụm 5-6**: 9 styles để chọn.
- **Cụm 7**: cách document.
- **Cụm 8**: áp end-to-end vào case thực.

Bạn đã có toolkit đầy đủ. Bước tiếp theo là *practice* trên dự án thật của bạn. Architecture là kỹ năng *applied*: chỉ đọc không đủ.

Suggested next steps:

1. Đọc lại Course Summary để consolidate.
2. Apply framework vào 1 dự án thật bạn đang làm.
3. Đọc thêm: *Fundamentals of Software Architecture* (Mark Richards), *Software Architecture in Practice* (Bass-Clements-Kazman).
4. Practice qua exam checklist nếu chuẩn bị phỏng vấn.

Chúc bạn trở thành software architect xuất sắc.